Dependencies with yield

FastAPI supports dependencies that do some *extra steps after finishing*.

To do this, use `yield` instead of `return`, and write the extra steps (code) after.

Tip

Make sure to use `yield` one single time per dependency.

Technical Details

Any function that is valid to use with:

- `@contextlib.contextmanager` or
- `@contextlib.asynccontextmanager`

would be valid to use as a **FastAPI** dependency.

In fact, FastAPI uses those two decorators internally.

A database dependency with `yield`

For example, you could use this to create a database session and close it after finishing.

Only the code prior to and including the `yield` statement is executed before creating a response:

Python 3.10+

```python
async def get_db():
    db = DBSession()
    try:
        yield db
    finally:
        db.close()
```

The yielded value is what is injected into *path operations* and other dependencies:

Python 3.10+

```python
async def get_db():
    db = DBSession()
    try:
        yield db
    finally:
        db.close()
```

The code following the `yield` statement is executed after the response:

Python 3.10+

```
async def get_db():
    db = DBSession()
    try:
        yield db
    finally:
        db.close()
```

Tip

You can use `async` or regular functions.

FastAPI will do the right thing with each, the same as with normal dependencies.

A dependency with `yield` and `try`

If you use a `try` block in a dependency with `yield`, you'll receive any exception that was thrown when using the dependency.

For example, if some code at some point in the middle, in another dependency or in a *path operation*, made a database transaction "rollback" or created any other exception, you would receive the exception in your dependency.

So, you can look for that specific exception inside the dependency with `except SomeException`.

In the same way, you can use `finally` to make sure the exit steps are executed, no matter if there was an exception or not.

Python 3.10+

```
async def get_db():
    db = DBSession()
    try:
        yield db
    finally:
        db.close()
```

Sub-dependencies with `yield`

You can have sub-dependencies and "trees" of sub-dependencies of any size and shape, and any or all of them can use `yield`.

FastAPI will make sure that the "exit code" in each dependency with `yield` is run in the correct order.

For example, `dependency_c` can have a dependency on `dependency_b`, and `dependency_b` on `dependency_a`:

Python 3.10+

```
from typing import Annotated

from fastapi import Depends


async def dependency_a():
    dep_a = generate_dep_a()
    try:
        yield dep_a
    finally:
        dep_a.close()


async def dependency_b(dep_a: Annotated[DepA, Depends(dependency_a)]):
```

```
    dep_b = generate_dep_b()
    try:
        yield dep_b
    finally:
        dep_b.close(dep_a)


async def dependency_c(dep_b: Annotated[DepB, Depends(dependency_b)]):
    dep_c = generate_dep_c()
    try:
        yield dep_c
    finally:
        dep_c.close(dep_b)
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```
from fastapi import Depends


async def dependency_a():
    dep_a = generate_dep_a()
    try:
        yield dep_a
    finally:
        dep_a.close()


async def dependency_b(dep_a=Depends(dependency_a)):
    dep_b = generate_dep_b()
    try:
        yield dep_b
    finally:
        dep_b.close(dep_a)


async def dependency_c(dep_b=Depends(dependency_b)):
    dep_c = generate_dep_c()
    try:
        yield dep_c
    finally:
        dep_c.close(dep_b)
```

And all of them can use `yield`.

In this case `dependency_c`, to execute its exit code, needs the value from `dependency_b` (here named `dep_b`) to still be available.

And, in turn, `dependency_b` needs the value from `dependency_a` (here named `dep_a`) to be available for its exit code.

Python 3.10+

```
from typing import Annotated

from fastapi import Depends


async def dependency_a():
```

```
    dep_a = generate_dep_a()
    try:
        yield dep_a
    finally:
        dep_a.close()


async def dependency_b(dep_a: Annotated[DepA, Depends(dependency_a)]):
    dep_b = generate_dep_b()
    try:
        yield dep_b
    finally:
        dep_b.close(dep_a)


async def dependency_c(dep_b: Annotated[DepB, Depends(dependency_b)]):
    dep_c = generate_dep_c()
    try:
        yield dep_c
    finally:
        dep_c.close(dep_b)
```

Other versions and variants

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```
from fastapi import Depends


async def dependency_a():
    dep_a = generate_dep_a()
    try:
        yield dep_a
    finally:
        dep_a.close()


async def dependency_b(dep_a=Depends(dependency_a)):
    dep_b = generate_dep_b()
    try:
        yield dep_b
    finally:
        dep_b.close(dep_a)


async def dependency_c(dep_b=Depends(dependency_b)):
    dep_c = generate_dep_c()
    try:
        yield dep_c
    finally:
        dep_c.close(dep_b)
```

The same way, you could have some dependencies with `yield` and some other dependencies with `return`, and have some of those depend on some of the others.

And you could have a single dependency that requires several other dependencies with `yield`, etc.

You can have any combinations of dependencies that you want.

FastAPI will make sure everything is run in the correct order.

Technical Details

This works thanks to Python's Context Managers.

FastAPI uses them internally to achieve this.

Dependencies with `yield` and `HTTPException`

You saw that you can use dependencies with `yield` and have `try` blocks that try to execute some code and then run some exit code after `finally`.

You can also use `except` to catch the exception that was raised and do something with it.

For example, you can raise a different exception, like `HTTPException`.

Tip

This is a somewhat advanced technique, and in most of the cases you won't really need it, as you can raise exceptions (including `HTTPException`) from inside of the rest of your application code, for example, in the *path operation function*.

But it's there for you if you need it. 🤓

Python 3.10+

```python
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException

app = FastAPI()


data = {
    "plumbus": {"description": "Freshly pickled plumbus", "owner": "Morty"},
    "portal-gun": {"description": "Gun to create portals", "owner": "Rick"},
}


class OwnerError(Exception):
    pass


def get_username():
    try:
        yield "Rick"
    except OwnerError as e:
        raise HTTPException(status_code=400, detail=f"Owner error: {e}")


@app.get("/items/{item_id}")
def get_item(item_id: str, username: Annotated[str, Depends(get_username)]):
    if item_id not in data:
        raise HTTPException(status_code=404, detail="Item not found")
    item = data[item_id]
    if item["owner"] != username:
        raise OwnerError(username)
    return item
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```python
from fastapi import Depends, FastAPI, HTTPException

app = FastAPI()


data = {
    "plumbus": {"description": "Freshly pickled plumbus", "owner": "Morty"},
    "portal-gun": {"description": "Gun to create portals", "owner": "Rick"},
}


class OwnerError(Exception):
    pass


def get_username():
    try:
        yield "Rick"
    except OwnerError as e:
        raise HTTPException(status_code=400, detail=f"Owner error: {e}")


@app.get("/items/{item_id}")
def get_item(item_id: str, username: str = Depends(get_username)):
    if item_id not in data:
        raise HTTPException(status_code=404, detail="Item not found")
    item = data[item_id]
    if item["owner"] != username:
        raise OwnerError(username)
    return item
```

If you want to catch exceptions and create a custom response based on that, create a Custom Exception Handler.

Dependencies with `yield` and `except`

If you catch an exception using `except` in a dependency with `yield` and you don't raise it again (or raise a new exception), FastAPI won't be able to notice there was an exception, the same way that would happen with regular Python:

Python 3.10+

```python
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException

app = FastAPI()


class InternalError(Exception):
    pass


def get_username():
    try:
```

```
        yield "Rick"
    except InternalError:
        print("Oops, we didn't raise again, Britney 😱")


@app.get("/items/{item_id}")
def get_item(item_id: str, username: Annotated[str, Depends(get_username)]):
    if item_id == "portal-gun":
        raise InternalError(
            f"The portal gun is too dangerous to be owned by {username}"
        )
    if item_id != "plumbus":
        raise HTTPException(
            status_code=404, detail="Item not found, there's only a plumbus here"
        )
    return item_id
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```
from fastapi import Depends, FastAPI, HTTPException

app = FastAPI()


class InternalError(Exception):
    pass


def get_username():
    try:
        yield "Rick"
    except InternalError:
        print("Oops, we didn't raise again, Britney 😱")


@app.get("/items/{item_id}")
def get_item(item_id: str, username: str = Depends(get_username)):
    if item_id == "portal-gun":
        raise InternalError(
            f"The portal gun is too dangerous to be owned by {username}"
        )
    if item_id != "plumbus":
        raise HTTPException(
            status_code=404, detail="Item not found, there's only a plumbus here"
        )
    return item_id
```

In this case, the client will see an *HTTP 500 Internal Server Error* response as it should, given that we are not raising an `HTTPException` or similar, but the server will **not have any logs** or any other indication of what was the error. 😱

Always `raise` in Dependencies with `yield` and `except`

If you catch an exception in a dependency with `yield`, unless you are raising another `HTTPException` or similar, **you should re-raise the original exception**.

You can re-raise the same exception using `raise`:

Python 3.10+

```python
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException

app = FastAPI()


class InternalError(Exception):
    pass


def get_username():
    try:
        yield "Rick"
    except InternalError:
        print("We don't swallow the internal error here, we raise again 😎")
        raise


@app.get("/items/{item_id}")
def get_item(item_id: str, username: Annotated[str, Depends(get_username)]):
    if item_id == "portal-gun":
        raise InternalError(
            f"The portal gun is too dangerous to be owned by {username}"
        )
    if item_id != "plumbus":
        raise HTTPException(
            status_code=404, detail="Item not found, there's only a plumbus here"
        )
    return item_id
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

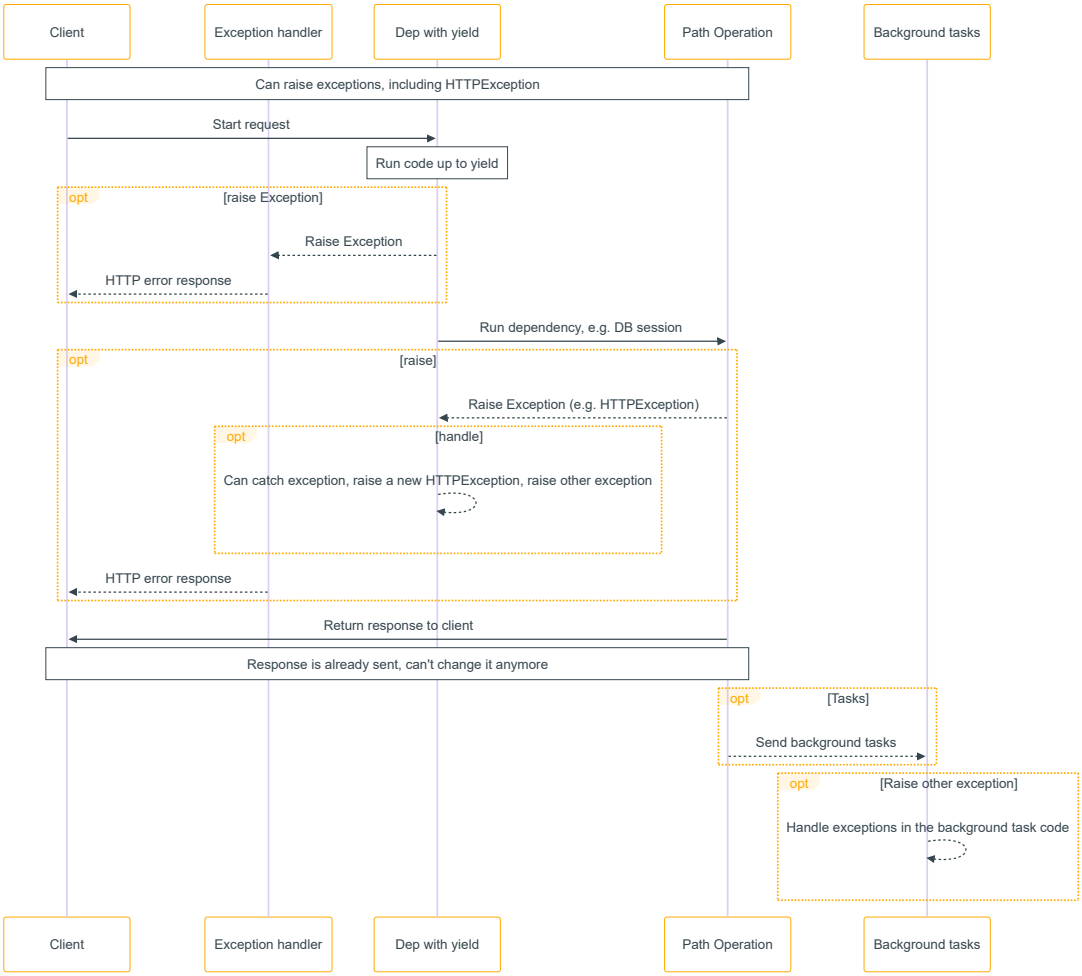
Prefer to use the `Annotated` version if possible.

```python
from fastapi import Depends, FastAPI, HTTPException

app = FastAPI()


class InternalError(Exception):
    pass


def get_username():
    try:
        yield "Rick"
    except InternalError:
        print("We don't swallow the internal error here, we raise again 😎")
        raise


@app.get("/items/{item_id}")
def get_item(item_id: str, username: str = Depends(get_username)):
    if item_id == "portal-gun":
        raise InternalError(
            f"The portal gun is too dangerous to be owned by {username}"
        )
    if item_id != "plumbus":
        raise HTTPException(
            status_code=404, detail="Item not found, there's only a plumbus here"
        )
    return item_id
```

Now the client will get the same *HTTP 500 Internal Server Error* response, but the server will have our custom `InternalError` in the logs. 😎

Execution of dependencies with `yield`

The sequence of execution is more or less like this diagram. Time flows from top to bottom. And each column is one of the parts interacting or executing code.



Info

Only **one response** will be sent to the client. It might be one of the error responses or it will be the response from the *path operation*.

After one of those responses is sent, no other response can be sent.

Tip

If you raise any exception in the code from the *path operation function*, it will be passed to the dependencies with yield, including `HTTPException`. In most cases you will want to re-raise that same exception or a new one from the dependency with `yield` to make sure it's properly handled.

Early exit and `scope`

Normally the exit code of dependencies with `yield` is executed **after the response** is sent to the client.

But if you know that you won't need to use the dependency after returning from the *path operation function*, you can use `Depends(scope="function")` to tell FastAPI that it should close the dependency after the *path operation function* returns, but **before** the **response is sent**.

Python 3.10+

```python
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()


def get_username():
    try:
        yield "Rick"
    finally:
        print("Cleanup up before response is sent")


@app.get("/users/me")
def get_user_me(username: Annotated[str, Depends(get_username, scope="function")]):
    return username
```

🤓 **Other versions and variants**

Python 3.10+ - non-Annotated

Tip

Prefer to use the `Annotated` version if possible.

```python
from fastapi import Depends, FastAPI

app = FastAPI()


def get_username():
    try:
        yield "Rick"
    finally:
        print("Cleanup up before response is sent")


@app.get("/users/me")
def get_user_me(username: str = Depends(get_username, scope="function")):
    return username
```

`Depends()` receives a `scope` parameter that can be:

- `"function"`: start the dependency before the *path operation function* that handles the request, end the dependency after the *path operation function* ends, but **before** the response is sent back to the client. So, the dependency function will be executed **around** the *path operation* ***function***.
- `"request"`: start the dependency before the *path operation function* that handles the request (similar to when using `"function"`), but end **after** the response is sent back to the client. So, the dependency function will be executed **around** the **request** and response cycle.

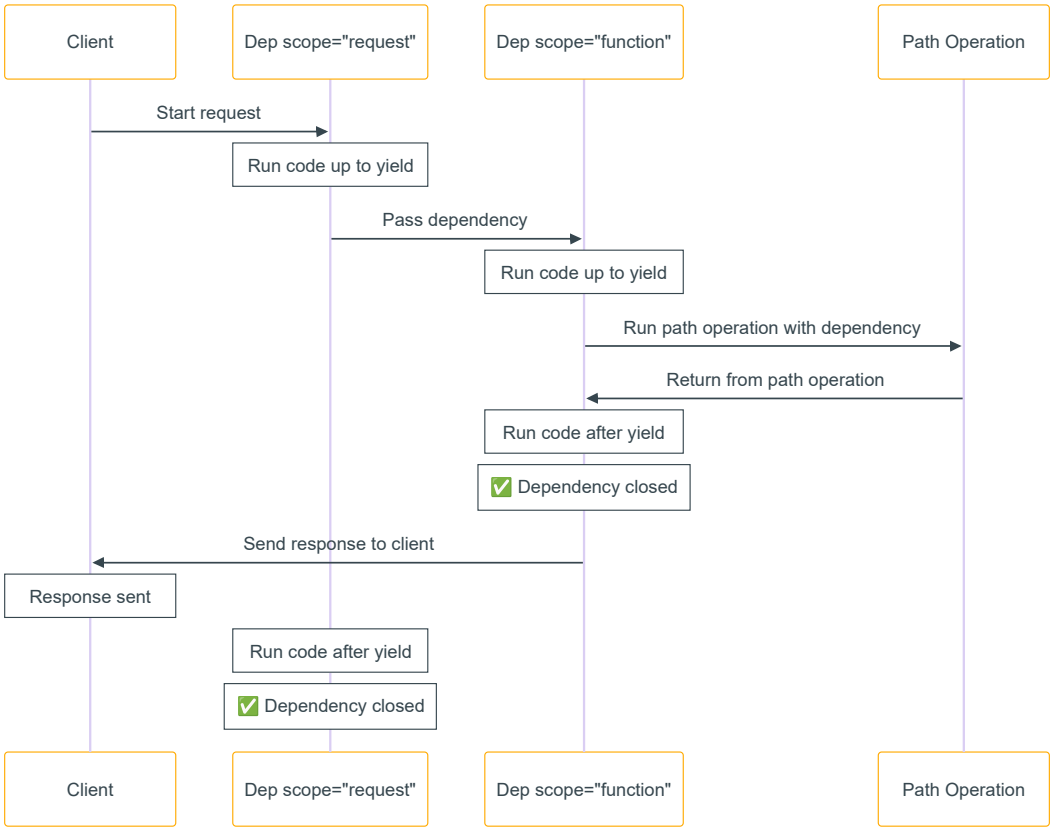
If not specified and the dependency has `yield`, it will have a `scope` of `"request"` by default.

`scope` for sub-dependencies

When you declare a dependency with a `scope="request"` (the default), any sub-dependency needs to also have a `scope` of `"request"`.

But a dependency with `scope` of `"function"` can have dependencies with `scope` of `"function"` and `scope` of `"request"`.

This is because any dependency needs to be able to run its exit code before the sub-dependencies, as it might need to still use them during its exit code.



Dependencies with `yield`, `HTTPException`, `except` and Background Tasks

Dependencies with `yield` have evolved over time to cover different use cases and fix some issues.

If you want to see what has changed in different versions of FastAPI, you can read more about it in the advanced guide, in Advanced Dependencies - Dependencies with `yield`, `HTTPException`, `except` and Background Tasks.

Context Managers

What are "Context Managers"

"Context Managers" are any of those Python objects that you can use in a `with` statement.

For example, you can use `with` to read a file:

```
with open("./somefile.txt") as f:
    contents = f.read()
    print(contents)
```

Underneath, the `open("./somefile.txt")` creates an object that is called a "Context Manager".

When the `with` block finishes, it makes sure to close the file, even if there were exceptions.

When you create a dependency with `yield`, **FastAPI** will internally create a context manager for it, and combine it with some other related tools.

Using context managers in dependencies with `yield`

> **Warning**
>
> This is, more or less, an "advanced" idea.
>
> If you are just starting with **FastAPI** you might want to skip it for now.

In Python, you can create Context Managers by creating a class with two methods: `__enter__()` and `__exit__()`.

You can also use them inside of **FastAPI** dependencies with `yield` by using `with` or `async with` statements inside of the dependency function:

Python 3.10+

```python
class MySuperContextManager:
    def __init__(self):
        self.db = DBSession()

    def __enter__(self):
        return self.db

    def __exit__(self, exc_type, exc_value, traceback):
        self.db.close()


async def get_db():
    with MySuperContextManager() as db:
        yield db
```

> **Tip**
>
> Another way to create a context manager is with:
>
> - `@contextlib.contextmanager` or
> - `@contextlib.asynccontextmanager`
>
> using them to decorate a function with a single `yield`.
>
> That's what **FastAPI** uses internally for dependencies with `yield`.
>
> But you don't have to use the decorators for FastAPI dependencies (and you shouldn't).
>
> FastAPI will do it for you internally.